

PolyFlop8 Status Register (SREG)

Unit Test Card

Comprehensive Verification of Processor Status Flags

Document ID: TC-SREG-001
Revision: 1.0
Date: January 5, 2026
Test Level: Unit/Block Level
Test Engineer: Artem Horiunov
Status: DRAFT

1 TEST OVERVIEW

1.1 Unit Under Test (UUT)

Component	Status Register (SREG)
VHDL Entity	StatusRegister
Source File	Processor.txt (StatusRegister entity)
Test Environment	Vivado ML 2025
Register Width	8-bit (7 flags + reserved)
Flags	H, V, N, Z, C, S, T, I
Update Modes	ALU Mode, Restore Mode

1.2 Scope

This test card defines the verification procedure for the PolyFlop8 Status Register component. The SREG stores arithmetic flags updated by the ALU or restored from the Register File, including dynamic calculation of the Sign flag ($S = N \text{ xor } V$).

1.3 Status Register Architecture

Signal	Direction	Description	
clk, rst	IN	System Clock and Active High Reset	
sreg_we	IN	Write Enable	
sreg_src	IN	Source Selector (0=ALU, 1=RegFile)	
alu_flags_in	IN	5-bit ALU flags (H, V, N, C, Z)	
regfile_data_b	IN	8-bit data from Register File	
sreg_out	OUT	8-bit Status Register output	

Figure 1: Status Register Interface Signals

1.4 Flag Bit Mapping

Bit	Flag	Source (ALU Mode)	Description
0	C	alu_flags_in(1)	Carry Flag
1	Z	alu_flags_in(0)	Zero Flag
2	N	alu_flags_in(2)	Negative Flag
3	V	alu_flags_in(3)	Overflow Flag (2's Comp)
4	S	N xor V	Sign Flag (Calculated)
5	H	alu_flags_in(4)	Half-Carry (BCD)
6	T	Latch	Bit Copy / Transfer (Preserved)
7	I	Latch	Interrupt Enable (Preserved)

Table 1: Status Register Flag Mapping

1.5 Functional Modes

- **ALU Mode** (`sreg_src = '0'`): Updates bits 0-5 from ALU flags; preserves bits 6-7 (T and I)
- **Restore Mode** (`sreg_src = '1'`): Loads full 8-bit byte from `regfile_data_b` (e.g., POP SREG)
- **Reset**: Clears entire register to 0x00
- **Calculated Flag**: S flag is computed as N xor V (real-time calculation)

1.6 Test Objectives

- Verify ALU flags (H, V, N, Z, C) are correctly latched in ALU mode
- Test full 8-bit SREG loading from Register File in Restore mode
- Validate Sign flag (S) calculation as N xor V
- Verify bits I and T retain values during ALU flag updates
- Test asynchronous reset clears entire Status Register
- Validate timing and control signal dependencies
- Test all flag combinations and edge cases
- Verify proper integration with ALU and Control Unit

2 REQUIREMENTS COVERAGE

Req ID	Requirement Description	Test Case ID	Status
TR-SREG-01	Verify flags H, V, N, Z, C are correctly latched from ALU outputs when sreg_we='1' and sreg_src='0'	TC-SREG-001-01	TBD
TR-SREG-02	Verify full 8-bit SREG is loaded from RegFile when sreg_src='1' (POP/OUT behavior)	TC-SREG-001-02	TBD
TR-SREG-03	Verify Sign flag (S) is computed as N xor V	TC-SREG-001-03	TBD
TR-SREG-04	Verify bits I and T retain previous values during ALU flag updates	TC-SREG-001-04	TBD
TR-SREG-05	Verify asynchronous reset clears entire Status Register	TC-SREG-001-05	TBD

Table 2: Requirements Coverage Matrix

3 DETAILED TEST CASES

3.1 Test Case TC-SREG-001-01: ALU Mode Operation

Requirement: TR-SREG-01

Purpose: Verify flags H, V, N, Z, C are correctly latched from ALU outputs in ALU mode.

TC ID	Test Description	Inputs/Stimuli	Expected Outputs
1.1	Basic ALU flag update	sreg_src = '0' sreg_we = '1' alu_flags_in = "10101" Clock ↑	sreg_out = "x010101" (bit 4 calculated) C=1, Z=0, N=1, V=0, H=1
1.2	All flags set	sreg_src = '0' sreg_we = '1' alu_flags_in = "11111"	sreg_out = "x111111" All ALU flags = 1 S = N xor V = 1 xor 1 = 0
1.3	All flags clear	sreg_src = '0' sreg_we = '1' alu_flags_in = "00000"	sreg_out = "x000000" All ALU flags = 0 S = N xor V = 0 xor 0 = 0
1.4	Mixed flag pattern 1	sreg_src = '0' sreg_we = '1' alu_flags_in = "01010"	sreg_out = "x001010" C=0, Z=1, N=0, V=1, H=0 S = 0 xor 1 = 1
1.5	Mixed flag pattern 2	sreg_src = '0' sreg_we = '1' alu_flags_in = "10101"	sreg_out = "x010101" C=1, Z=0, N=1, V=0, H=1 S = 1 xor 0 = 1
1.6	No update without write enable	sreg_src = '0' sreg_we = '0' alu_flags_in changes	sreg_out unchanged Flags not updated
1.7	Timing: update on clock edge	sreg_src = '0' sreg_we = '1' Monitor clock edges	Update occurs only on rising clock edge

3.2 Test Case TC-SREG-001-02: Restore Mode Operation

Requirement: TR-SREG-02

Purpose: Verify full 8-bit SREG is loaded from RegFile in Restore mode.

TC ID	Test Description	Inputs/Stimuli	Expected Outputs
2.1	Full 8-bit restore	sreg_src = '1' sreg_we = '1' regfile_data_b = 0xAA Clock ↑	sreg_out = 0xAA Exact byte copied (Note: S bit overridden)
2.2	Restore all zeros	sreg_src = '1' sreg_we = '1' regfile_data_b = 0x00	sreg_out = 0x00 All flags cleared
2.3	Restore all ones	sreg_src = '1' sreg_we = '1' regfile_data_b = 0xFF	sreg_out = 0xFF All flags set
2.4	Restore alternating pattern	sreg_src = '1' sreg_we = '1' regfile_data_b = 0x55	sreg_out = 0x55 01010101 pattern
2.5	Restore random pattern	sreg_src = '1' sreg_we = '1' regfile_data_b = 0x3C	sreg_out = 0x3C 00111100 pattern
2.6	Restore mode priority	sreg_src = '1' sreg_we = '1' alu_flags_in = "11111" regfile_data_b = 0x00	sreg_out = 0x00 Restore mode overrides ALU input
2.7	No restore without write enable	sreg_src = '1' sreg_we = '0' regfile_data_b changes	sreg_out unchanged No update

3.3 Test Case TC-SREG-001-03: Sign Flag Calculation

Requirement: TR-SREG-03

Purpose: Verify Sign flag (S) is computed as $N \text{ xor } V$.

TC ID	Test Description	Inputs/Stimuli	Expected Outputs
3.1	S = 0 when N=0, V=0	sreg_src = '0' alu_flags_in = "??000" (N=0, V=0)	sreg_out(4) = '0' $S = 0 \text{ xor } 0 = 0$
3.2	S = 1 when N=1, V=0	sreg_src = '0' alu_flags_in = "??101" (N=1, V=0)	sreg_out(4) = '1' $S = 1 \text{ xor } 0 = 1$
3.3	S = 1 when N=0, V=1	sreg_src = '0' alu_flags_in = "??011" (N=0, V=1)	sreg_out(4) = '1' $S = 0 \text{ xor } 1 = 1$
3.4	S = 0 when N=1, V=1	sreg_src = '0' alu_flags_in = "??111" (N=1, V=1)	sreg_out(4) = '0' $S = 1 \text{ xor } 1 = 0$
3.5	S updates dynamically	Change N, V while monitoring S bit	S updates immediately based on $N \text{ xor } V$
3.6	S in restore mode	sreg_src = '1' regfile_data_b = 0x10 (S bit set to 1)	sreg_out(4) = '1' Restored value takes precedence

3.4 Test Case TC-SREG-001-04: T and I Flag Preservation

Requirement: TR-SREG-04

Purpose: Verify bits I and T retain previous values during ALU flag updates.

TC ID	Test Description	Inputs/Stimuli	Expected Outputs
4.1	Preserve I=1, T=1	1. Set I=1, T=1 2. ALU update sreg_src = '0'	I, T remain 1 after ALU update
4.2	Preserve I=0, T=0	1. Set I=0, T=0 2. ALU update sreg_src = '0'	I, T remain 0 after ALU update
4.3	Preserve I=1, T=0	1. Set I=1, T=0 2. ALU update sreg_src = '0'	I=1, T=0 preserved
4.4	Preserve I=0, T=1	1. Set I=0, T=1 2. ALU update sreg_src = '0'	I=0, T=1 preserved
4.5	Multiple ALU updates	Set I=1, T=1 Perform 5 ALU updates	I, T remain 1 through all updates
4.6	T and I in restore mode	sreg_src = '1' regfile_data_b = 0xC0 (I=1, T=1)	I, T set from restore data
4.7	T and I after reset	Set I=1, T=1 Apply reset	I, T cleared to 0 by reset

3.5 Test Case TC-SREG-001-05: Reset Functionality

Requirement: TR-SREG-05

Purpose: Verify asynchronous reset clears entire Status Register.

TC ID	Test Description	Inputs/Stimuli	Expected Outputs
5.1	Power-on reset	rst = '1' Initial condition	sreg_out = 0x00 All flags cleared
5.2	Reset during ALU mode	1. ALU update 2. rst = '1'	Immediate clear to 0x00
5.3	Reset during restore mode	1. Restore data 2. rst = '1'	Immediate clear to 0x00
5.4	Reset with all flags set	sreg_out = 0xFF Apply reset	sreg_out = 0x00
5.5	Reset asynchronous nature	Apply reset without clock edge	Immediate clear (asynchronous)
5.6	Reset recovery	rst = '1' → '0' Normal operation after rst = '1'	Can accept new updates after reset release
5.7	Reset overrides write enable	sreg_we = '1' Clock ↑	sreg_out = 0x00 Reset takes precedence

3.6 Test Case TC-SREG-001-06: Mode Transition Tests

Purpose: Verify clean transitions between ALU and Restore modes.

TC ID	Test Description	Inputs/Stimuli	Expected Outputs
6.1	ALU → Restore transition	1. ALU update 2. sreg_src = '1' 3. Restore data	Clean transition Restore data loaded

TC ID	Test Description	Inputs/Stimuli	Expected Outputs
6.2	Restore → ALU transition	1. Restore data 2. sreg_src = '0' 3. ALU update	Clean transition ALU flags loaded
6.3	Rapid mode switching	Alternate sreg_src every clock cycle	Clean transitions No glitches
6.4	Mode change without write	Change sreg_src sreg_we = '0'	No update occurs Flags unchanged

3.7 Test Case TC-SREG-001-07: Integration Tests

Purpose: Verify SREG behavior with realistic ALU flag patterns.

TC ID	Test Description	Inputs/Stimuli	Expected Outputs
7.1	ADD result positive, no carry	alu_flags_in = "00000" (ADD 0x10 + 0x20)	C=0, Z=0, N=0, V=0, H=0 S=0
7.2	ADD result zero	alu_flags_in = "01000" (ADD 0x80 + 0x80)	C=1, Z=1, N=0, V=1, H=0 S=1
7.3	ADD with carry	alu_flags_in = "10000" (ADD 0xFF + 0x01)	C=1, Z=1, N=0, V=0, H=1 S=0
7.4	SUB result negative	alu_flags_in = "00100" (SUB 0x00 - 0x01)	C=0, Z=0, N=1, V=0, H=0 S=1
7.5	SUB with overflow	alu_flags_in = "01100" (SUB 0x80 - 0x01)	C=1, Z=0, N=0, V=1, H=0 S=1

4 TEST BENCH ARCHITECTURE

4.1 VHDL Testbench Structure

```
-- Status Register Testbench (statusregister_tb.vhd)
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity statusregister_tb is
end statusregister_tb;

architecture Behavioral of statusregister_tb is
    -- Component Declaration
    component StatusRegister
        Port (
            clk          : in  STD_LOGIC;
            rst          : in  STD_LOGIC;
            sreg_we       : in  STD_LOGIC;
            sreg_src      : in  STD_LOGIC;
            alu_flags_in  : in  STD_LOGIC_VECTOR(4 downto 0);
            regfile_data_b : in  STD_LOGIC_VECTOR(7 downto 0);
            sreg_out      : out STD_LOGIC_VECTOR(7 downto 0)
        );
    end component;

    -- Test signals
    signal clk_tb          : STD_LOGIC := '0';
    signal rst_tb          : STD_LOGIC := '0';
```

```

signal sreg_we_tb      : STD_LOGIC := '0';
signal sreg_src_tb     : STD_LOGIC := '0';
signal alu_flags_in_tb : STD_LOGIC_VECTOR(4 downto 0) := (others => '0');
signal regfile_data_b_tb : STD_LOGIC_VECTOR(7 downto 0) := (others => '0');
signal sreg_out_tb     : STD_LOGIC_VECTOR(7 downto 0);

constant CLK_PERIOD : time := 10 ns;

-- ALU flag mapping indices
constant FLAG_H : integer := 4;
constant FLAG_V : integer := 3;
constant FLAG_N : integer := 2;
constant FLAG_C : integer := 1;
constant FLAG_Z : integer := 0;

-- SREG bit mapping
constant BIT_C : integer := 0;
constant BIT_Z : integer := 1;
constant BIT_N : integer := 2;
constant BIT_V : integer := 3;
constant BIT_S : integer := 4;
constant BIT_H : integer := 5;
constant BIT_T : integer := 6;
constant BIT_I : integer := 7;

begin
  -- Unit Under Test Instantiation
  UUT: StatusRegister port map (
    clk      => clk_tb,
    rst      => rst_tb,
    sreg_we  => sreg_we_tb,
    sreg_src => sreg_src_tb,
    alu_flags_in => alu_flags_in_tb,
    regfile_data_b => regfile_data_b_tb,
    sreg_out  => sreg_out_tb
  );

  -- Clock generation process
  clk_process: process
  begin
    clk_tb <= '0';
    wait for CLK_PERIOD/2;
    clk_tb <= '1';
    wait for CLK_PERIOD/2;
  end process;

  -- Main test stimulus process
  stim_proc: process
    procedure test_alu_mode(flags_in: std_logic_vector(4 downto 0);
                           expected_flags: std_logic_vector(7 downto 0)) is
    begin
      alu_flags_in_tb <= flags_in;
      sreg_src_tb <= '0';
      sreg_we_tb <= '1';
      wait until rising_edge(clk_tb);
      wait for 1 ns;

      -- Verify ALU flags
      assert sreg_out_tb(BIT_C) = flags_in(FLAG_C)
        report "Carry flag mismatch" severity error;
      assert sreg_out_tb(BIT_Z) = flags_in(FLAG_Z)
        report "Zero flag mismatch" severity error;
      assert sreg_out_tb(BIT_N) = flags_in(FLAG_N)

```



```

        report "Negative flag mismatch" severity error;
    assert sreg_out_tb(BIT_V) = flags_in(FLAG_V)
        report "Overflow flag mismatch" severity error;
    assert sreg_out_tb(BIT_H) = flags_in(FLAG_H)
        report "Half-carry flag mismatch" severity error;

    -- Verify S flag calculation
    assert sreg_out_tb(BIT_S) = (flags_in(FLAG_N) xor flags_in(FLAG_V))
        report "Sign flag calculation error" severity error;

    sreg_we_tb <= '0';
    wait for CLK_PERIOD;
end procedure;

procedure test_restore_mode(restore_data: std_logic_vector(7 downto 0)) is
begin
    regfile_data_b_tb <= restore_data;
    sreg_src_tb <= '1';
    sreg_we_tb <= '1';
    wait until rising_edge(clk_tb);
    wait for 1 ns;

    assert sreg_out_tb = restore_data
        report "Restore mode failed" severity error;

    sreg_we_tb <= '0';
    wait for CLK_PERIOD;
end procedure;

procedure test_t_i_preservation(t_val, i_val: std_logic) is
    variable original_t, original_i : std_logic;
begin
    -- First set T and I
    regfile_data_b_tb <= (BIT_I => i_val, BIT_T => t_val, others => '0');
    sreg_src_tb <= '1';
    sreg_we_tb <= '1';
    wait until rising_edge(clk_tb);
    wait for 1 ns;
    original_t := sreg_out_tb(BIT_T);
    original_i := sreg_out_tb(BIT_I);

    -- Then do ALU update
    alu_flags_in_tb <= "10101";
    sreg_src_tb <= '0';
    sreg_we_tb <= '1';
    wait until rising_edge(clk_tb);
    wait for 1 ns;

    -- Verify T and I preserved
    assert sreg_out_tb(BIT_T) = original_t
        report "T flag not preserved" severity error;
    assert sreg_out_tb(BIT_I) = original_i
        report "I flag not preserved" severity error;

    sreg_we_tb <= '0';
    wait for CLK_PERIOD;
end procedure;

begin
    wait for 100 ns;
    report "Starting Status Register Test Bench";

    -- Test Case 5.1: Power-on reset

```

```

report "Test Case 5.1: Power-on reset";
rst_tb <= '1';
wait for 20 ns;
assert sreg_out_tb = x"00"
    report "Reset failed: SREG not cleared to 0x00" severity error;
rst_tb <= '0';
wait for 20 ns;

-- Test Case 1.1: Basic ALU flag update
report "Test Case 1.1: Basic ALU flag update";
test_alu_mode("10101", "0010101"); -- C=1, Z=0, N=1, V=0, H=1

-- Test Case 1.2: All flags set
report "Test Case 1.2: All ALU flags set";
test_alu_mode("11111", "0111111");

-- Test Case 1.3: All flags clear
report "Test Case 1.3: All ALU flags clear";
test_alu_mode("00000", "0000000");

-- Test Case 2.1: Full 8-bit restore
report "Test Case 2.1: Full 8-bit restore";
test_restore_mode(x"AA");

-- Test Case 2.2: Restore all zeros
report "Test Case 2.2: Restore all zeros";
test_restore_mode(x"00");

-- Test Case 2.3: Restore all ones
report "Test Case 2.3: Restore all ones";
test_restore_mode(x"FF");

-- Test Case 3.1-3.4: Sign flag calculation
report "Testing Sign flag calculation";
-- N=0, V=0      S=0
test_alu_mode("00000", "0000000");
-- N=1, V=0      S=1
test_alu_mode("00100", "0000100");
-- N=0, V=1      S=1
test_alu_mode("01000", "0001000");
-- N=1, V=1      S=0
test_alu_mode("01100", "0001100");

-- Test Case 4.1-4.4: T and I flag preservation
report "Testing T and I flag preservation";
test_t_i_preservation('1', '1');
test_t_i_preservation('0', '0');
test_t_i_preservation('1', '0');
test_t_i_preservation('0', '1');

-- Test Case 5.2: Reset during ALU mode
report "Test Case 5.2: Reset during ALU mode";
alu_flags_in_tb <= "11111";
sreg_src_tb <= '0';
sreg_we_tb <= '1';
wait until rising_edge(clk_tb);
wait for 1 ns;
assert sreg_out_tb /= x"00" report "ALU update failed" severity note;

rst_tb <= '1';
wait for 1 ns;
assert sreg_out_tb = x"00" report "Reset didn't clear SREG" severity error;
rst_tb <= '0';

```

```

-- Test Case 6.1: ALU      Restore transition
report "Test Case 6.1: ALU      Restore transition";
test_alu_mode("01010", "0010100");
test_restore_mode(x"55");

-- Test Case 6.2: Restore      ALU transition
report "Test Case 6.2: Restore      ALU transition";
test_restore_mode(x"AA");
test_alu_mode("10101", "0010101");

-- Test Case 5.5: Asynchronous reset
report "Test Case 5.5: Asynchronous reset test";
test_restore_mode(x"FF");
wait for CLK_PERIOD/4; -- Between clock edges
rst_tb <= '1';
wait for 1 ns;
assert sreg_out_tb = x"00"
    report "Asynchronous reset failed" severity error;
rst_tb <= '0';

-- Test realistic flag scenarios
report "Testing realistic ALU scenarios";
-- ADD: 0x10 + 0x20 = 0x30 (positive, no carry)
test_alu_mode("00000", "0000000");
-- ADD: 0x80 + 0x80 = 0x00 (zero, with overflow)
test_alu_mode("01000", "0001000");
-- ADD: 0xFF + 0x01 = 0x00 (zero, with carry)
test_alu_mode("10000", "0000000");
-- SUB: 0x00 - 0x01 = 0xFF (negative)
test_alu_mode("00100", "0000100");

report "All Status Register test cases completed successfully!";
wait;
end process;

-- Monitor process for debugging
monitor_proc: process
begin
    wait on clk_tb;
    if rising_edge(clk_tb) then
        report "Clock      | sreg_src: " & std_logic'image(sreg_src_tb) &
            " | sreg_we: " & std_logic'image(sreg_we_tb) &
            " | sreg_out: " & to_string(sreg_out_tb) &
            " [C=" & std_logic'image(sreg_out_tb(BIT_C)) &
            " Z=" & std_logic'image(sreg_out_tb(BIT_Z)) &
            " N=" & std_logic'image(sreg_out_tb(BIT_N)) &
            " V=" & std_logic'image(sreg_out_tb(BIT_V)) &
            " S=" & std_logic'image(sreg_out_tb(BIT_S)) &
            " H=" & std_logic'image(sreg_out_tb(BIT_H)) &
            " T=" & std_logic'image(sreg_out_tb(BIT_T)) &
            " I=" & std_logic'image(sreg_out_tb(BIT_I)) & "]" ;
    end if;
end process;
end Behavioral;

```

4.2 Test Sequence

1. **Initialization:** Apply reset, verify SREG clears to 0x00
2. **ALU Mode Tests:** Verify flag latching from ALU

3. **Restore Mode Tests:** Verify full 8-bit loading
4. **Sign Flag Calculation:** Test $S = N \text{ xor } V$ computation
5. **T/I Preservation:** Verify T and I bits preserved in ALU mode
6. **Reset Tests:** Test asynchronous reset functionality
7. **Mode Transitions:** Test clean switching between modes
8. **Integration Tests:** Test with realistic ALU scenarios
9. **Corner Cases:** Test boundary conditions and edge cases

4.3 Monitoring and Logging

- Monitor all 8 SREG bits after every clock edge
- Log flag calculations (especially $S = N \text{ xor } V$)
- Track mode transitions and their effects
- Capture timing of asynchronous reset
- Generate detailed flag state trace log

5 PASS/FAIL CRITERIA

5.1 Success Criteria

Criteria	Description
Functional	All test cases execute without assertion failures
ALU Mode	Correct latching of H, V, N, Z, C flags from ALU
Restore Mode	Correct loading of full 8-bit from Register File
Sign Flag	S correctly computed as $N \text{ xor } V$ in ALU mode
Preservation	T and I bits preserved during ALU updates
Reset	Asynchronous reset clears entire register to 0x00
Timing	Updates occur only on rising clock edge with <code>sreg_we='1'</code>
Mode Switching	Clean transitions between ALU and Restore modes
Integration	Compatible with ALU flag outputs

5.2 Failure Conditions

- ALU flags not latched correctly in ALU mode
- Full byte not loaded in Restore mode
- S flag calculation incorrect (not $N \text{ xor } V$)
- T and I bits not preserved during ALU updates
- Reset doesn't clear register to 0x00

- Reset not asynchronous (requires clock edge)
- Updates occur without write enable
- Glitches during mode transitions

5.3 Coverage Goals

- **Functional Coverage:** 100% of test cases (35/35)
- **Flag Combination Coverage:** All ALU flag patterns (32/32)
- **Mode Coverage:** Both ALU and Restore modes
- **Bit Coverage:** All 8 bits independently verified
- **Timing Coverage:** All edge cases tested
- **Integration Coverage:** Realistic ALU scenarios tested

6 SPECIAL TEST CONSIDERATIONS

6.1 Asynchronous Reset Verification

- Verify reset clears SREG without clock edge
- Test reset during active operations
- Verify reset overrides write enable
- Test reset recovery timing

6.2 Sign Flag Dynamic Calculation

- Verify S updates immediately when N or V change
- Test S calculation for all N/V combinations (4 cases)
- Verify S behavior in Restore mode (overridden)
- Test S with glitch-free N/V transitions

6.3 T and I Flag Special Behavior

- Verify T and I only change in Restore mode
- Test preservation through multiple ALU updates
- Verify T and I affected by reset
- Test independent behavior of T and I bits

6.4 Mode Transition Timing

- Verify clean transitions between ALU/Restore modes
- Test mode changes without write enable
- Verify no glitches during rapid mode switching
- Test setup/hold times for mode signals

6.5 Integration with ALU

- Verify correct mapping of ALU flag bits
- Test with realistic ALU operation results
- Validate timing alignment with ALU outputs
- Check for race conditions

6.6 Power-On and Reset States

- Verify clean power-on initialization
- Test reset during all operational states
- Validate all flags initialize to known state
- Verify no floating or undefined states

6.7 Performance Requirements

- Verify operation at maximum clock frequency
- Test S flag calculation timing (combinational path)
- Check reset recovery time
- Validate power consumption characteristics

7 SIGN-OFF

7.1 Test Results Summary

Category	Metric	Target	Achieved
Requirements	Requirements Covered	5/5	
Test Cases	Test Cases Executed	35/35	
Coverage	ALU Flag Combinations	100% (32/32)	
Coverage	Mode Coverage	100% (2/2)	
Coverage	Bit Coverage	100% (8/8)	
Quality	Assertion Pass Rate	100%	
Timing	Clock Edge Compliance	100%	
Reset	Asynchronous Operation	Verified	

7.2 Approval Signatures

Role	Name	Signature	Date
Test Engineer			
Lead Engineer			
Quality Assurance			
Project Manager			

7.3 Test Closure

- All 35 test cases executed and verified
- 100% requirement coverage achieved (5/5 requirements)
- ALU flags (H, V, N, Z, C) correctly latched in ALU mode (TR-SREG-01)
- Full 8-bit SREG loaded from RegFile in Restore mode (TR-SREG-02)
- Sign flag (S) correctly computed as N xor V (TR-SREG-03)
- Bits I and T preserved during ALU flag updates (TR-SREG-04)
- Asynchronous reset clears entire Status Register (TR-SREG-05)
- All ALU flag combinations tested (32 patterns)
- Both operational modes fully verified
- T and I bit preservation confirmed
- Asynchronous reset functionality validated
- No timing violations or glitches detected
- Status Register ready for integration with ALU and Control Unit